

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF03:03:Systemarchitekturen

1) Client-Server-Model [→ ZP:Sheet:2]

- Das **Server**-
 - *programm ...*
 - * läuft auf dem Serverrechner.
 - * lauscht dort auf einen Socket: Bei dessen Initialisierung hat es dem Betriebssystem bekanntgegeben, für welche Port-Anfragen der Server zuständig sein will.
 - * liest eingehende Nachrichten vom Socket.
 - * schreibt eine Antwort auf den Socket
 - *betriebssystem*
 - * schreibt eine an seiner Netzwerkschnittstelle eingehende Anfrage - spezifiziert durch IP-Adresse (des Servers) und Portnummer - auf den entsprechenden Socket.
 - * liest die Antwort des **Server**programms vom Socket und schreibt sie - geeignet verpackt - auf das Netzwerkinterface
- Das **Client**-
 - *programm*
 - * läuft auf dem Clientrechner
 - * schreibt seine Anfrage, die URL und Port des Serverrechners enthält, auf einen temporären Socket
 - * lauscht anschließend am temporären Socket auf eine Antwort
 - *betriebssystem*
 - * liest die Anfrage samt Ziel-URL und Port vom temporären Socket
 - * schreibt die Anfrage im geeigneten Format auf die Netzwerkkarte: unter Angabe der Quell-IP-Adresse (Layer III) und Quell-Port (Layer IV) und Ziel-IP-Adresse (des Servers) und Ziel-Portnummer
 - * schreibt eine auf der Netzwerkkarte eingehende Nachricht mit Quell-Port der Anfrage als Ziel-Port auf den temporären Port

Dieses Client-Server-Model - abstrahiert von architekturellen Details - fokussiert sich auf die Request(= Client)-Response(= Server)-Struktur.

Grundsätzlich können Client- und Serversoftware können auf demselben Rechner laufen oder auf verschiedenen:

1.A) Client-Server-Many-2-One-Model [→ ZP:Sheet:3]

Bei Client-Server-Architekturen (mit Hardwareausdifferenzierung) = Client-Server-Many-2-One-Architekturen

- laufen Client- und Serversoftware auf verschiedenen Rechnern
- Loadbalancer verteilen die Requests auf verschiedene Server.

1.B) Client-Server-Many-2-Many-Model [→ ZP:Sheet:4]

Bei Peer-To-Peer-Architekturen (mit Hardwareausdifferenzierung) = Client-Server-Many-2-Many-Architekturen

- ist jeder Rechner für jeden anderen gleichermaßen Client und Server

2) Cloud-Architekturen [→ ZP:Sheet:5]

Jede Cloud * besteht auf der untersten Ebene aus Rechnern (Plattenstacks), die mit Betriebssystem und Virtualisierer ausgestattet sind * hat eine Virtualisierungsschicht, die - zusammen mit der Orchestrierungssoftware - virtuelle Maschinen auf einer realen Maschine designt, startet, stoppt und zerstört. * virtuelle Maschinen mit je eigenem Betriebssystem und mehr oder minder komplexe Softwarekomponenten.

Je nach Fertigungstiefe liefert der Cloudbetreiber unterschiedliche Vorprodukte:

- **Infrastructure as a Service** = alles, was zum Aufsetzen von virtuellen Maschinen mit Betriebssystem und SSH-Zugang nötig ist.
- **Data(base) as a Service** = alles, was zum Aufsetzen von virtuellen Maschinen mit Betriebssystem und Datenbanken nötig ist.
- **Platform as a Service** = alles, was zum Aufsetzen von virtuellen Maschinen mit Betriebssystem und einer Plattformsoftware nötig ist.
- **Software as a Service** = alles, was zum Aufsetzen von virtuellen Maschinen mit Betriebssystem und einer fertig nutzbaren Software nötig ist.

Bei *Public Clouds* und *Private Clouds* werden Nutzergruppen unterschieden; das Cloudkonzept als solches bleibt gleich.

Sonderfälle [→ ZP:Sheet:6]

verlegt die Berechnung von Daten von der zentralen Cloud physisch näher an die Nutzer der berechneten Daten, um die Latenzzeit bei der Übertragung zu verringern.

- **Cloud Computing** =

- Zentralisierte Clouds berechnen aus vorgehaltenen Daten Ergebnisse und bringen sie **über die volle Netzstrecke** zum Nutzer.
- Die Sichtweise kann auch umgekehrt sein: Die an der Netzwerkkante neu erhobenen Daten werden gleich an der Grenze zum Netzwerk verarbeitet, anstatt erst über die ganze Strecke in die zentralisierte Cloud gebracht zu werden.

- **Edge Computing** =

- Lokale Clouds nahe der Netzwerkgrenze (= “network’s edge”) berechnen aus lokal vorgehaltenen Daten Ergebnisse und bringen sie **über eine möglichst geringe Reststrecke** zum Nutzer.

- **Fog Computing** = Mehrere Computer berechnen - **verteilt über die ganze Netzstrecke** - berechnen aus erhaltenen Daten Zwischen(Ergebnisse) und senden sie zum nächsten Knoten.

- https://en.wikipedia.org/wiki/Edge_computing

Gemeinsamkeiten

Zentrales Merkmal einer Cloud bleibt die **Virtualisierung**: wo sie in welcher Größe betrieben wird, ist sekundär.

Bei der Virtualisierung spricht man von Hosts und Guests (Virtual Machines). Es braucht immer einen Host, auf dem die Virtualisierungssoftware läuft:

- **Emulator** = Software, der Systemaufrufe aus anderen Betriebssystemen im Client auf Systemaufrufe des Hostsystems abbildet (Atari auf Linux)
- **Paravirtualisierung** = liefert eine hardwarenahe abstrakte Verwaltungsschicht (= Hypervisor), auf der sich mehrere VMs die Hardware teilen. Im Extremfall durch einen *bare-metal-hypervisor*, der so hardwarenah läuft, dass keine Hostbetriebssystem mehr nötig ist. (Linux/KVM, Linux/Xen VMware...)
- **Virtualisierung auf Betriebssystemebene** =
 - Mehrere voneinander isolierte Umgebungen werden auf einem einzigen Kernel gestartet.
 - Sie teilen sich Hardware Ressourcen

Vorläufer

Sieht man vom Aspekt der Virtualisierung ab, sind die Konzepte *Thin-Client* und *Remote-Desktop* **Cloud-Vorläufer**

• Thin-Client

- physisch schwach ausgerüsteten Rechner
- erhält fertig vorgerechnete Daten vom Server
- stellt diese dar
- erfasst User-Reaktionen darauf (Mausklicks, Tastatureingaben)
- schickt diese zum Server zurück.

• Remote-Desktop-System

- ein *Server* (Host)
 - * hostet alle Applikationen,
 - * führt sie samt Darstellungsschicht aus,
 - * schickt das darstellbare Ergebnis (als Bilder) zu einem Client
- ein *Client*
 - * erfasst User-Reaktionen darauf (Mausklicks, Tastatureingaben)
 - * schickt diese zum Server zurück.

In beiden Fällen wird der Vorteil einer einfacheren Pflege der rechenintensiven Programme auf dem stark ausgerüsteten Server bei im Prinzip nur schwach ausgerüsteten Server durch den Nachteil des erhöhten Netztraffics und einer größeren Latenz erkaufte.